



PROBLEM STATEMENT LETTER



Academic Project Collaboration

SovereignAI

On-Premise Agentic Multimodal AI Workbench

CodeNiq Software Development, Pune, Maharashtra • In Collaboration With CloudSEK

To: Mr. Suyash Sanjay Hadole

Date:
21-09-2026

Subject: Industry Problem Statement and Academic Project Collaboration – SovereignAI

Dear Mr. Suyash Hadole,

This document presents a proposed industry-oriented problem statement for the development of **SovereignAI – On-Premise Agentic Multimodal AI Workbench**. The proposed project integrates **Generative AI, Agentic AI, Multimodal AI, Retrieval-Augmented Generation (RAG), OCR, local AI inference, Data Engineering, and secure automation** to support confidential enterprise knowledge work.

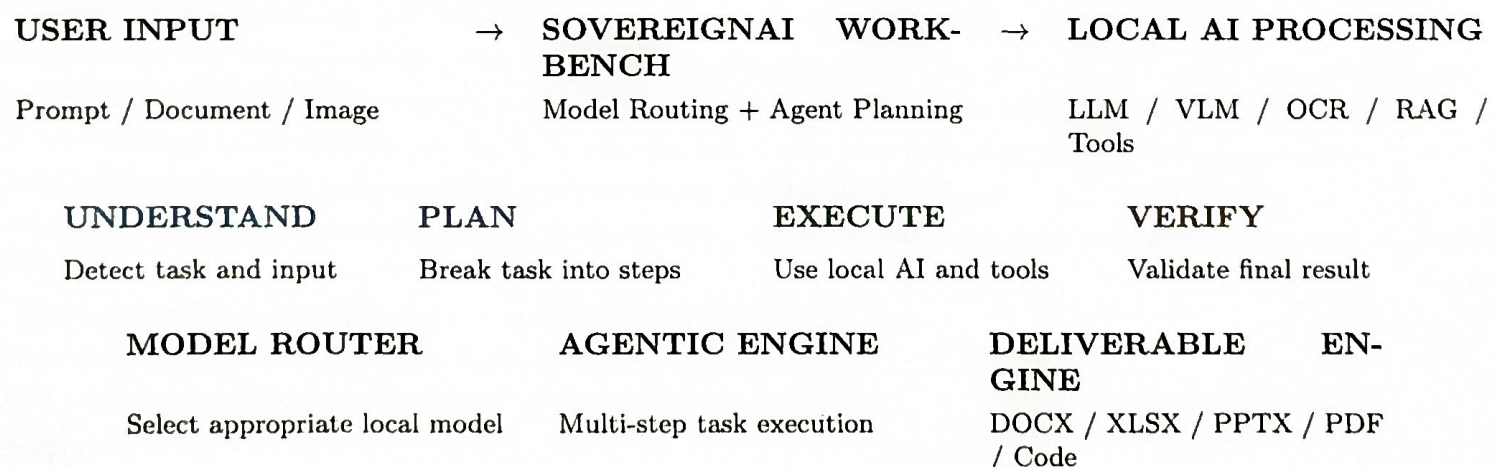
INDUSTRY PROBLEM STATEMENT

Organizations increasingly work with confidential documents, technical reports, engineering drawings, scanned records, spreadsheets, internal procedures, business documents, and other sensitive information. Conventional cloud-based Generative AI workflows may require confidential information to be processed outside the organization's controlled infrastructure.

The proposed problem is to develop a secure, self-hosted AI workbench that can process confidential documents and multimodal inputs locally, automatically select suitable AI models, perform multi-step Agentic AI workflows, retrieve information from a private knowledge base, execute controlled tools, verify results, and generate practical business deliverables without relying on external AI services for confidential processing.

Proposed Solution

SovereignAI will function as an intelligent local AI workspace between the user, confidential data, AI models, knowledge sources, tools, and generated outputs. It will understand the user's task, route the task to an appropriate local model, plan multi-step actions, retrieve relevant information, process documents and images, execute tools, verify results, and generate usable deliverables.



The proposed platform is intended to demonstrate the practical integration of **Generative AI, Agentic AI, Multimodal AI, RAG, OCR, local inference, secure tool execution, and automated deliverable generation** within a unified enterprise application.

The platform will be designed to remain **model-agnostic**, allowing different open-weight models to be added or replaced without redesigning the complete application.

Regards,

CodeniQ Team

ANNEXURE – A

SovereignAI – Project Scope and Technical Specification

1. Objectives

- Develop a self-hosted AI workbench for confidential enterprise information.
- Support multiple local open-weight AI models with automatic task-based model selection.
- Implement Agentic AI capable of planning and executing multi-step workflows.
- Process text, PDFs, scanned documents, images, engineering drawings, and other multimodal inputs.
- Implement a local knowledge base using Retrieval-Augmented Generation (RAG).
- Provide local tools for file processing, calculations, spreadsheets, document processing, and code execution.
- Generate practical deliverables such as Word, Excel, PowerPoint, PDF, and code files.
- Provide secure sandbox execution for AI-generated code.
- Maintain execution history, audit logs, and model/tool activity information.
- Keep confidential AI processing within the organization's controlled infrastructure.

2. Core Functional Modules

Module	Purpose
Task Management	Accept natural-language tasks and associated documents or files.
Model Router	Classify the task and automatically select a suitable local AI model.
Agent Orchestrator	Plan and execute multi-step workflows using local models and tools.
Multimodal Processor	Process text, scanned documents, images, engineering drawings, and mixed text-image content.
OCR Engine	Extract text from scanned PDFs, photographs, and image-based documents.
Knowledge Base / RAG	Retrieve relevant information from private manuals, SOPs, policies, and technical documentation.
Tool Engine	Provide controlled tools for file operations, calculations, spreadsheets, and structured data processing.
Secure Sandbox	Execute generated code inside an isolated environment.
Deliverable Generator	Generate DOCX, XLSX, PPTX, PDF, reports, and source-code files.
Verification Engine	Check execution outputs and generated results before completion.
Security / Audit Monitor	Record model usage, agent actions, tool calls, file operations, and network activity.
Dashboard	Display tasks, documents, models, agent status, outputs, and security information.

3. AI and Data Engineering Components

- **Generative AI:** Local open-weight LLMs for reasoning, summarization, drafting, transformation, and natural-language interaction.
- **Agentic AI:** Task planning, tool selection, iterative execution, observation, and result verification.
- **Multimodal AI:** Vision-language processing for images, scanned documents, drawings, and mixed text-image inputs.
- **OCR:** Local optical character recognition for scanned PDFs, photographs, and image-based records.
- **RAG:** Retrieval of relevant information from a locally maintained knowledge base before response generation.
- **Embeddings:** Semantic representation of documents for efficient local retrieval.
- **Data Engineering:** Ingestion, cleaning, transformation, chunking, indexing, metadata management, and structured storage.

4. Expected Inputs

- Natural-language prompts and task instructions.
- PDF and scanned PDF documents.
- Word and spreadsheet files.
- Engineering drawings and photographs.
- Handwritten or image-based documents.
- Manuals, SOPs, policies, and technical documentation.
- CSV and other structured datasets.

5. Proposed Technology Stack

Area	Suggested Technologies
Programming	Python, TypeScript
AIML / Deep Learning	PyTorch, Scikit-learn
NLP	Hugging Face Transformers
Generative AI	Open-weight LLMs, local inference frameworks
Multimodal AI	Vision-Language Models
GenAI / RAG	Sentence Transformers, FAISS / Qdrant
OCR	PaddleOCR / Tesseract
Data Engineering	Pandas, NumPy
Database	PostgreSQL / SQLite
Backend	FastAPI
Frontend	React, TypeScript
Document Processing	PyMuPDF, python-docx, openpyxl, python-pptx
Sandbox	Docker
Deployment	Local Workstation / GPU Server

ANNEXURE – B
Confidentiality, Data Access, Security and Intellectual Property

6. Confidentiality

All confidential project and organization-related information shall be handled only within the approved project scope. Confidential documents, technical reports, source code, credentials, proprietary data, model artifacts, internal procedures, and organization-specific knowledge shall not be unnecessarily disclosed to external services.

7. Data Access and Usage

Project data shall be accessed only by authorized users and used for approved development, testing, demonstration, and academic purposes.

Sensitive or organization-specific information should be stored using appropriate controls and anonymized or sanitized where required.

No production credential, authentication token, secret, restricted dataset, or internal security configuration shall be hard-coded into the application.

8. Security Requirements

- Secure input validation and file handling.
- Authentication and access control for protected workspaces.
- Role-based access for administrative functions where required.
- Sandboxed execution of generated code.
- Controlled access to documents and knowledge-base data.
- Audit logging of important AI, agent, tool, and file operations.
- Local processing of sensitive project information.
- Monitoring of outbound network activity during secure/offline operation.
- Separation of development, testing, and demonstration data.
- Secure handling of generated files and tool outputs.

9. Local AI and Network Policy

The system is designed around local inference using open-weight AI models running on the organization's own workstation or server.

Confidential processing should remain inside the controlled deployment environment rather than being sent to external cloud AI APIs.

The application should provide a visible security or audit mechanism showing the status of local AI services, agent operations, tool calls, and network activity.

10. Project Deliverables

1. Working SovereignAI prototype.
2. Multi-model local AI inference and model-routing layer.
3. Agentic workflow and tool-execution system.
4. Multimodal processing pipeline with OCR and vision support.
5. Local RAG knowledge base.
6. Secure code-execution sandbox.
7. Automated DOCX, XLSX, PPTX, PDF, and code generation.
8. Security and audit dashboard.
9. End-to-end demonstration workflows.
10. Technical documentation, system architecture, testing evidence, final report, and presentation.

11. Representative Workflow

INPUT	→	AI PROCESSING	→	OUTPUT
Inspection Report		OCR + RAG + Agent + Model Router		Approval Note + Summary
IMAGE / DRAWING	→	VISION + AGENT	→	ANALYSIS / REPORT
Engineering Drawing		Visual Understanding + Tools		Structured Explanation
CSV / DATA	→	CODING MODEL + SAND-BOX	→	VERIFIED RESULT
Structured Dataset		Generate + Execute Code		Analysis + Report

INDUSTRY COLLABORATION AND AUTHORIZATION

**PROPOSED PROBLEM STATEMENT**

Develop a secure, self-hosted Agentic AI Workbench that can understand confidential documents and multimodal inputs, automatically select appropriate local AI models, use a private knowledge base and controlled tools, perform multi-step AI workflows, verify results, and generate practical deliverables while keeping sensitive processing within the organization's controlled infrastructure.

Project Classification

Domain: Generative AI / Agentic AI / Multimodal AI / RAG / AML / Data Engineering
Project Type: Software-only academic and industry-oriented prototype
Outcome: SovereignAI web workbench, local AI models, model router, Agentic AI engine, RAG knowledge base, multimodal processing pipeline, secure sandbox, deliverable generator, and security/audit dashboard
Deployment: Self-hosted / on-premise / controlled local environment

FOR CODENIQ SOFTWARE DEVELOPMENT**Authorized Representative:**

Samar Manoj Sharma

Signature / Company Seal:**Designation:**

Founder

Date: 01-09-2026